QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

1. Scope & Requirements Traceability 3
2. Risk Assessment & Mitigation Strategy 4
3. Test Environment & Execution Strategy 6
4. CI/CD Release Thresholds & Automation Gates 7
5. Test Case Specifications (Drafts) 9
6. AI Output & Boundary Testing (Drafts)S 11
11

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	UM Hackathon 2026 - SyncBite AI (Team 4NewBee)
Team Repo URL	https://github.com/ChinwenTan/SmartStock-AI.git
Project Board URL	—
Live Deployment URL (optional for preliminary round)	https://smart-stock-ai-beryl.vercel.app/

Objective:

The primary objective is to ensure that SyncBite-AI reliably processes merchant inventory inputs, accurately integrates live campus context signals, and consistently delivers sound AI-generated business strategy recommendations, and successfully prompts the Z.AI GLM to generate profitable pivot strategies without timeouts or hallucinations

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

1.1 In-Scope Core Features

- **Inventory Management:** Manual data entry, automated risk-tier classification (Safe, Medium, High).
- **Campus Intelligence:** Calendar synchronization, campus selection, anomaly detection (e.g., weather alerts).
- **Z.AI Strategy Engine:** Prompt construction, GLM inference, payload parsing (Action, Reasoning, Impact).
- **AI Assistant Chat:** Processing text, image, and voice inputs via the chatbox UI.

1.2 Out-of-Scope

- Direct POS system or delivery platform integration
- Automated execution of promotions without merchant approval
- Multi-user or team account management
- Real-time social media sentiment scraping
- Live university ERP or calendar API integration

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. Risk Assessment & Mitigation Strategy

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
GLM returns a hallucinated discount price below the merchant cost price	*3*	*5*	*15 (High)*	*Strict output schema enforcement (Action, Reasoning, Impact). Backend validates that the recommended price never falls below inputted cost price field*	*Inject test inventory with cost price RM5.00 and verify GLM never recommends a selling price below RM5.00*
Z.AI API timeout or complete unresponsiveness	*3*	*4*	*12 (High)*	*Auto-retry loop up to 3 attempts with escalating delay. If all retries fail, get_mock_advice() fallback is triggered*	*Force API timeout via network throttling and verify fallback strategy is displayed within an acceptable time*
Campus calendar displays incorrect or outdated condition	*2*	*4*	*8 (Medium)*	*Calendar data is manually curated and colour-coded. No live API dependency in current version eliminates external failure risk*	*Manually verify calendar rendering accuracy against known academic dates for linked universities*
Merchant inputs incorrect cost or selling price	*4*	*4*	*16(Critical)*	*UI enforces numeric-only input fields with mandatory validation. Financial impact predictions are clearly labelled as estimates*	*Submit zero-value, negative, and non-numeric inputs and verify form validation blocks submission*

Risk Assessment Scoring Criteria:

Likelihood (1-5)		*Severity (1-5)*	
1	*Rare*	*1*	*Impact is Negligible*
2	*Unlikely*	*2*	*Impact is Minor*
3	*Possible*	*3*	*Moderate Impact*

© UMHackathon 2026

Email: umhackathon@um.edu.my

4	*Likely*	*4*	*Major Impact*
5	*Almost Certain*	*5*	*Critical Failure of the system*

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	***Risk Level***	***Recommended Action***
1 – 5	*Low*	*Monitor only. Acceptable risks.*
6 – 10	*Medium*	*Mitigate + Testing*
11 – 15	*High*	*Must need mitigating, and thorough testing is required*
16 – 25	*Critical*	*Priority is Highest. Need an extensive level of testing.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Test Environment & Execution Strategy

This section defines where the testing of SyncBite-AI takes place, how we manage test data, and the rules governing our testing phases across our React frontend, FastAPI backend, and Z.AI integration.

- **Unit Test**
    - **Scope**: Inventory risk-tier calculations (shelf life math) and prompt payload construction.
    - **Execution**: Tests are written using PyTest for the backend. Execution happens locally during development and automatically upon pushing code to the repository.
    - **Isolation**: To focus only on our internal logic, external dependencies like the Z.AI API are mocked (simulated) so tests run fast and without internet.
    - **Pass Condition:** The math must be 100% accurate (e.g., an item expiring in 2 hours must strictly be flagged as "High Risk")
- **Integration Test**
    - **Scope**: FastAPI backend communication with the Z.AI model (ilmu-glm-5.1) and Vercel Frontend data submissions.
    - **Execution**: Performed after unit tests pass, using Postman for API route testing and manual testing on preview deployment links.
    - **Workflow**: Real API calls are made to Z.AI (without mocking) to ensure our prompt engineering successfully forces the GLM to return the correct JSON structure (Action, Reasoning, Impact).
    - **Pass Condition**: Integrated components must work together smoothly without 504 Gateway Timeouts, and the Z.AI response must perfectly populate the frontend UI cards.
- **Test Environment (CI/CD Practice):**
    - **Local Testing**: Manual testing done on localhost:5173 (React) and localhost:8000 (FastAPI).
    - **Staging/CI:** Vercel automatically generates a live "Preview URL" every time a Pull Request (PR) is created. This allows the team to test UI changes before merging.
    - **CI/CD Automated Pipeline**: GitHub Actions runs basic syntax and PyTest checks. Once approved, merging to the main branch triggers automatic production deployments on Vercel (Frontend) and Render (Backend).

- **Regression Testing & Pass/Fail Rules:**
    - **Execution Phase:** We test all 3 core features (Dashboard, Campus Sync, AI Strategy) every time a new feature branch is merged into the main branch to ensure nothing old was broken.
    - **Pass/Fail Condition**: A test only passes if the actual outcome matches the expected outcome. If the AI recommends a strategy that results in a negative profit margin, the test instantly fails and is logged as a critical defect.
    - **Continuation Rule:** Backend API routes and Z.AI payload generation must pass their thresholds before the team moves on to testing the Frontend UI elements.

- **Test Data Strategy:**
    - **Manual Data:** Created three mock merchant profiles (a Dessert Shop Owner, a Food Truck Operator, and a Canteen Operator). Each profile is loaded with different perishable inventory compositions and linked to different university campuses to manually test the

© UMHackathon 2026

Email: umhackathon@um.edu.my

UI and Campus Intelligence Hub.

- **Simulated Data (JSON Fixtures)**: Utilized hardcoded JSON (mock_inventory.json) payloads representing various external campus conditions (e.g., Normal Day, Exam Week, Heavy Rain, Public Holiday). These fixtures are injected into the backend to safely simulate GLM input scenarios and test the AI's reasoning without requiring a live, populated database.dummy data]

- **Passing Rate Threshold:**
  - **General Threshold:** A minimum of 85% of all general UI and user-flow test cases must pass.
  - **Critical Threshold:** A 100% pass rate is strictly required for critical features. There are no exceptions for the following:
    - The Profit Margin Guardrail (The AI must never recommend selling below cost price).
    - The API Timeout Fallback (The system must successfully trigger mock advice if Z.AI goes offline, ensuring the app never crashes for the user).

4. CI/CD Release Thresholds & Automation Gates

This section defines the strict metrics and thresholds required for success. If any of these thresholds are not met during the CI/CD pipeline, GitHub Actions will block the forward transition, preventing broken code from reaching our live Vercel (Frontend) and Render (Backend) deployments.

4.1 Integration Thresholds (Merging to Main)

The primary purpose of this threshold is to ensure the main branch remains a stable source of truth. Every Pull Request (PR) must pass these continuous integration checks before it is allowed to merge.

Checks	**Requirements**	**Project**	**Pass/Failed**
Automatic Build	**Zero Build Errors on React (Vite) & FastAPI**	**0 Errors**	**Passed**
Unit Tests	**100% Passing Rate for backend math (PyTest)**	**100%**	**Passed**
Code Quality	**Zero Linting Error**	**0 Errors**	**Passed**

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Coverage	**Minimum 80% coverage on core logic**	**86%**	**Passed**

© UMHackathon 2026

Email: umhackathon@um.edu.my

4.2 Deployment Thresholds (Pushing to Production)

Checks	Requirements	Project	Pass/Failed
Regression Test	**Minimum 90% pass rate across core UI**	**95%**	**Passed**
AI Output Pass Rate	**100% Pass Rate for Profit-Margin Safety Guardrails**	**100%**	**Passed**
Critical Bugs	**Zero P0/P1 Bugs (e.g., UI crashes or 504 errors)**	**0**	**Passed**
API Performance	**Response Time < 800ms**	**320ms**	**Passed**
Z.AI Inference Latency	**GLM payload processing & response < 5000ms**	**3150ms**	**Passed**
Security	**API keys (Z.AI) strictly hidden in .env**	**Clean**	**Passed**

5. Test Case Specifications (Drafts)

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	**Happy Case (Entire Flow):** Conversational AI Strategy	Verify that a merchant can successfully use the Multimodal Chat Assistant to log inventory, state a campus anomaly, and receive a complete,	1. Open the Chat Assistant interface. 2. User inputs context: "It's Final Exam week and students are	Order should The backend successfully parses the conversational context and queries the Z.AI engine. The	Successfully, It successfully linked the croissant inventory to the Exam Week context, generating a

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
		actionable business pivot strategy in a single conversational flow.	stuck studying in their dorms. I have 30 Butter Croissants in stock. How can I increase my sales today?" 3. Click "Send". 4. Review the AI response.	Chat UI returns a structured strategy including a specific Action (e.g., Delivery Bundle), Reasoning, and Expected Financial Recovery.	targeted "Study Fuel Care Package" promo for dorm delivery. Projected a 25% daily revenue boost. Status: Passed
TC-02	**Happy Case (Entire Flow):** Smart Inventory & Risk Math	Verify that a merchant can manually input a new perishable item, specifically logging its expiry date and time, and the backend correctly calculates the shelf-life to assign the accurate Risk Tier.	1. Navigate to "Add Inventory" on the Dashboard. 2. Input Name ("Butter Croissant"), Quantity (30), and Prices. 3. Input exact Expiry Date & Time (e.g., 1-2 days left).	The item saves successfully. The backend mathematical logic correctly calculates the time difference and accurately classifies the item into the "Medium Risk" tier (green badge).	The shelf-life math executed perfectly, calculating 48 hours remaining and accurately tagging the croissant with a green "Medium Risk" status badge on the UI. Status: Passed
TC-03	**Specific Case (Negative):** Profit Margin Guardrail	Verify the system prevents the AI from recommending a flash sale price that falls below the merchant's cost price, preventing guaranteed financial loss.	1. Input item: Cost Price RM5.00, Selling Price RM12.00. 2. Provide an extreme panic prompt (e.g., "Campus closed, force massive discount"). 3. Click "Generate	The backend validation blocks any AI output that suggests a price below RM5.00. UI displays a warning: "Strategy requires manual adjustment to protect margins."	AI hallucinated a price of RM4.50. Backend validation caught the error and blocked the UI update. Warning message displayed properly.

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
			Strategy".		Status: Passed
TC-04	**NFR (Performance):** API Timeout Fallback	Ensure the application degrades gracefully and does not crash the UI if the Z.AI API takes too long to respond or goes offline.	1. Disconnect the backend from the internet (or provide an invalid Z.AI API key). 2. Request a pivot strategy from the UI. 3. Wait for the 5-second timeout threshold.	UI shows a loading animation, then after 60s, displays a hardcoded mock fallback strategy . No 500 Internal Server Error is thrown.	Average Response time is Fallback triggered exactly at 130s. UI remained stable and displayed the emergency mock advice. App did not crash. Status: Failed
TC-05	**NFR (Load/Constraint):** Oversized Image Upload	Ensure the frontend correctly blocks oversized multimodal payloads from crashing the FastAPI backend or exceeding API token limits.	1. Open the Multimodal Chat Assistant. 2. Attempt to upload a 25MB high-resolution RAW photo of the inventory. 3. Click "Send".	The Vercel frontend catches the file size before sending it to Render. UI instantly displays: "File size exceeded. Please compress or upload a new photo."	Frontend successfully blocked the upload. Error message displayed immediately. The backend was not hit, saving bandwidth and tokens. Status: Passed

6. AI Output & Boundary Testing (Drafts)

This section is designed to validate that your GLM integration does produce acceptable output and handles any inputs that are abnormal gracefully.

© UMHackathon 2026

Email: umhackathon@um.edu.my

6.1. **Prompt/Response Test Pairs**

Document at least 3 Prompts/response test pairs. For each of them, do define the acceptance criteria - what a correct, acceptable or a failing response may look like for your use case.

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	"How can I increase my cafe sales?"	Output must provide actionable business strategies (e.g., bundles, loyalty programs, marketing) in a professional, structured tone.	The AI provided a 7-point strategic framework covering menu optimization, customer experience, social media marketing, loyalty programs, and operational efficiency.	Pass
AI-02	"What items can I prepare for tomorrow that are easy to sell in university?"	Output should categorize items (food/stationery) and suggest items suitable for student budgets.	The AI provided a comprehensive list categorized into Food & Drinks (e.g., Kuih-muih, Sandwiches), Stationery, and Essentials, including a "Tips for Success" section.	Pass
AI-03	“What is the weather tomorrow at 2 PM? Can I open my stall outside?”	AI should identify its lack of live data, refer to official sources, and provide safety-first advice for outdoor operations.	The AI correctly identified the limitation, directed the user to MetMalaysia / MyCuaca, and provided actionable safety advice for stall operations.	Pass

© UMHackathon 2026

Email: umhackathon@um.edu.my

6.2. Oversized/Larger Input Test

In this section, do define the maximum input size your system accepts. Do highlight here what happens when the limit is exceeded.

Fields	Details
Maximum Input Size	500 characters (approx. 70–80 words)
Input used while testing	2,500 characters (approx. 400 words – 5x the limit)
Expected Behavior	Reject input and display a user-friendly error message
Actual Behavior	System intercepted input, rejected the request, and returned a concise error message
Test Output	"That message is too long! Please keep it brief so I can help you better."
Status	Pass

6.3. Adversarial/Edge Prompt Test

- Test Prompt (Ambiguous/Malformed): "Stock, now. Yes? No. Maybe?"
- **GLM Response:** In raw tests, the GLM-5.1 model often tries to guess the user's intent, leading to a generic, non-actionable response like: "It seems like you're asking about stocks, but your question is a bit unclear! Could you help me understand what you'd like to know?" The model defaults to a financial market perspective rather than an inventory management perspective.

- **System Handling:** The SyncBite system implements a "Contextual Fallback" strategy. Instead of allowing the LLM to provide a generic financial disclaimer, the system triggers a default helper response: "I apologize, but I didn't quite catch that. Could you please specify which stock item or campaign you'd like to check? For example: 'What is the stock status of Iced Coffee?'"

- **Logic:** By validating the input length and keyword presence in main.py before sending the request to the OpenAI client, we ensure the LLM only processes structured or semi-structured data relevant to cafe operations. This pre-processing layer acts as a domain-filter, preventing the model from hallucinating or defaulting to off-topic financial advice.

6.4. Hallucinating Handling

© UMHackathon 2026

Email: umhackathon@um.edu.my

- **System Prompt Grounding:** The LLM is explicitly instructed: "Only provide responses based on the provided cafe inventory context. If the requested item or data point is not available, explicitly state that you do not have that information."

- **Deterministic Response Logic:** We limit the output length to < 60 words to prevent the model from entering a "generative loop" where it might start fabricating details to fulfill a long-form response.

- **Input Validation:** Our backend performs a sanity check on the length and structure of the input to ensure the model isn't processing "garbage" data that would increase the probability of a hallucinated output.

© UMHackathon 2026

Email: umhackathon@um.edu.my